



Dynamic Collaborative Scheduling System for New Energy Logistics Fleets Based on Graph Algorithms and Reinforcement Learning

Zepeng Lin, Guanyu Xin, Yuxuan Xue

Abstract

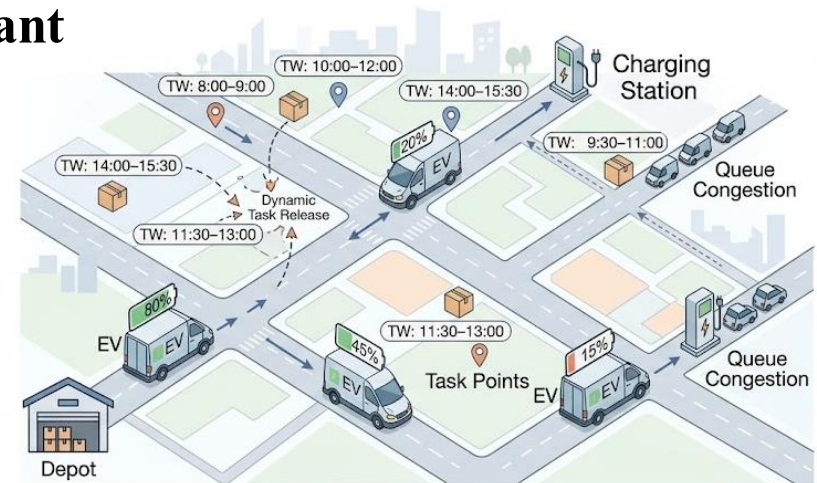
- 1. Background**
- 2. Problem setting**
- 3. Framework**
- 4. Experiment**

Background

INDUSTRY:

- rapid development of e-commerce/ instant delivery/smart logistics/urban delivery

➡ Timeliness and Cost Control.

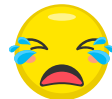


New Energy Vehicles



low emissions

high energy efficiency



Limited battery capacity

Long charging time

Scarce charging station

resources

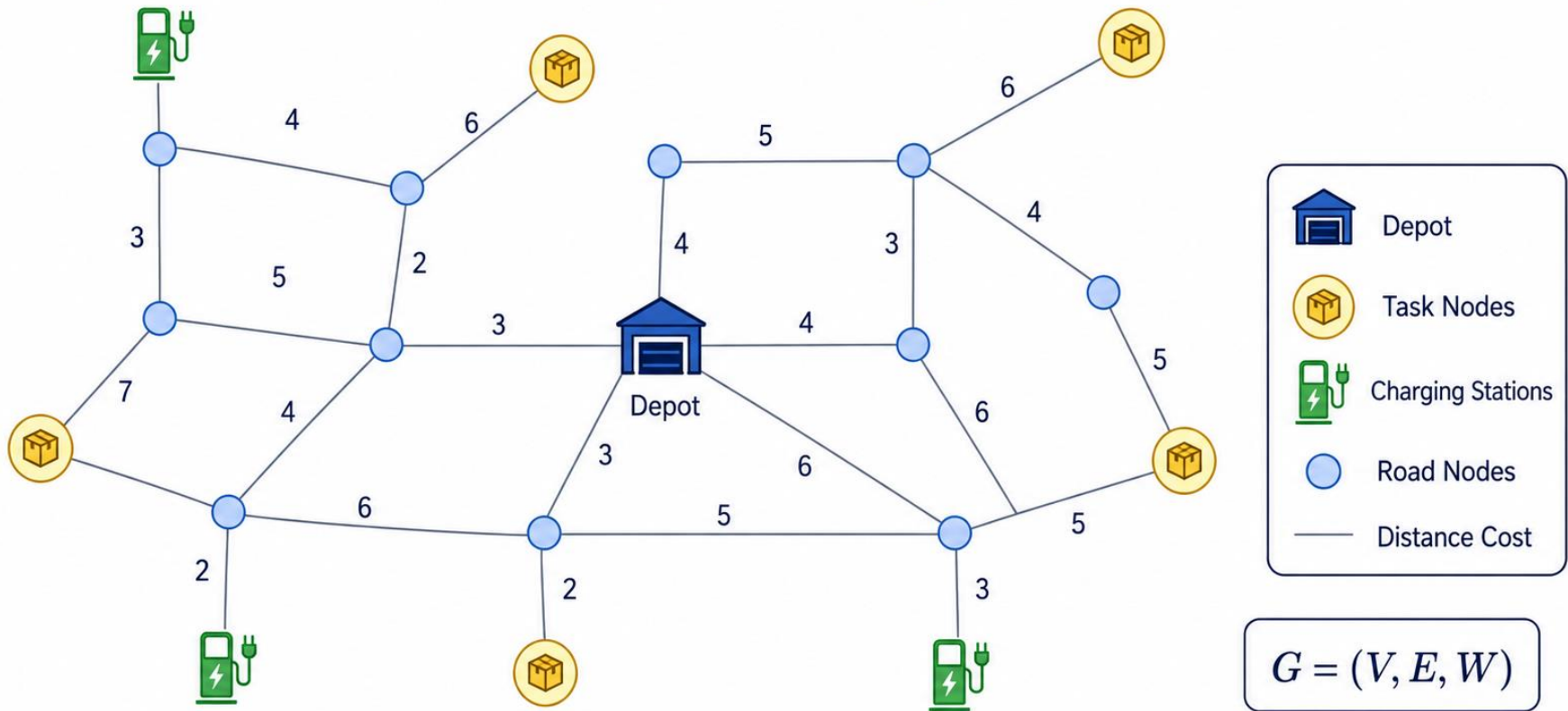


Problem Setting

5

Urban Road $\longrightarrow$ Weighted Graph

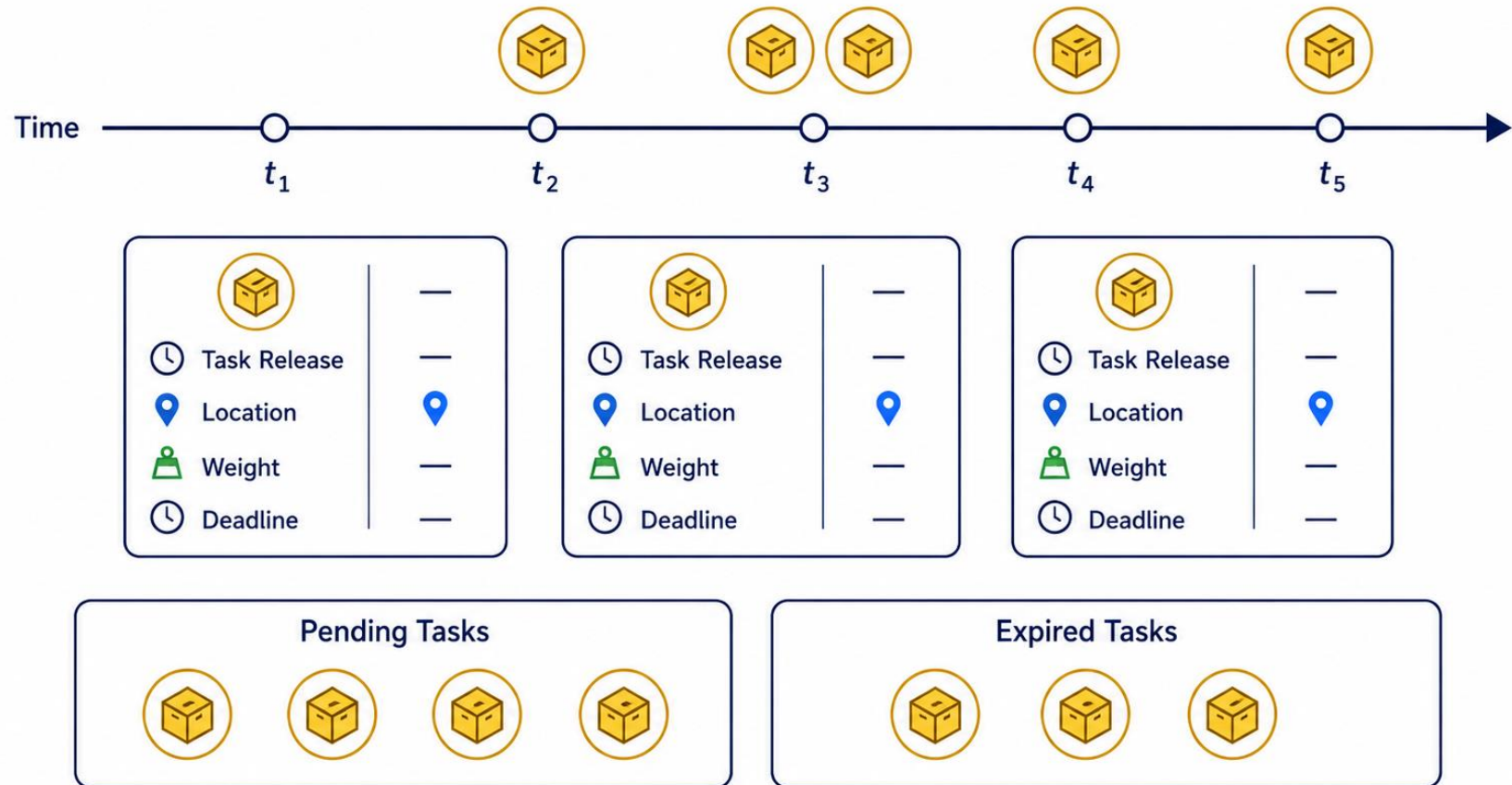
Weighted Road Graph



Problem Setting

6

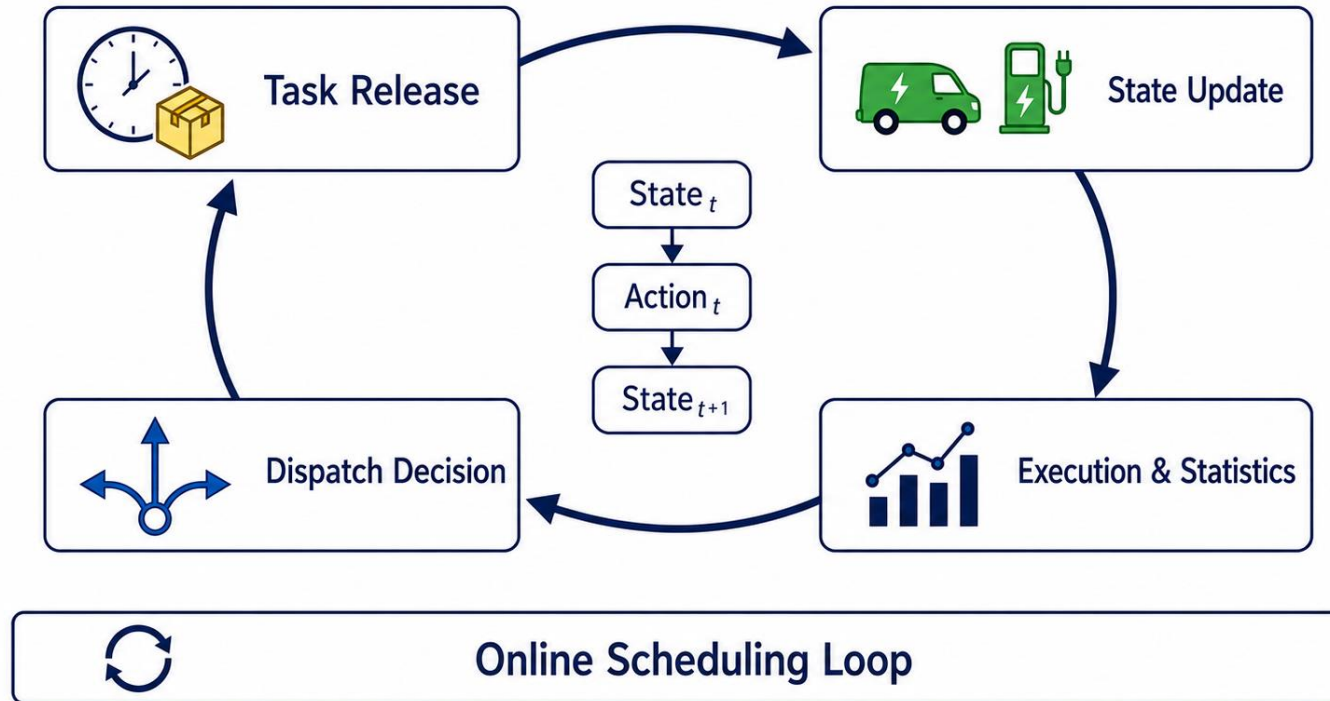
Dynamic Task



Problem Setting

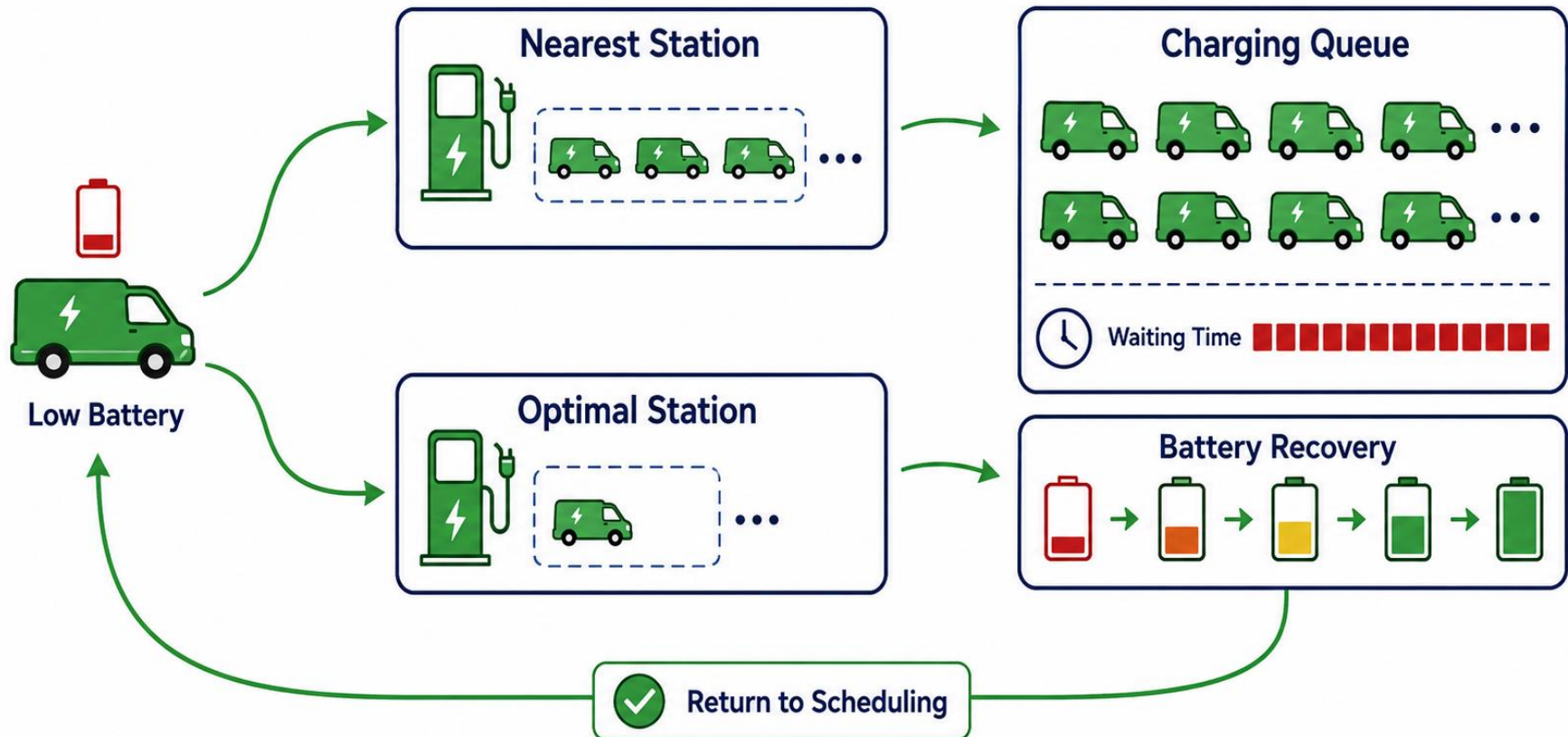
7

Online Closed-Loop Scheduling



Problem Setting

EV Car Modeling

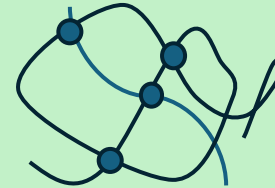


Problem Setting

Road Network

$$G = (V, E, W)$$

- V : task point/ charging station
- $E \subseteq V \times V$: Road connectivity
- $W: E \rightarrow \mathbf{R}^+$: Distance /Driving Danger Cost



Car



$$x_k = (b_k^{\{max\}}, b_k, c_k, v_k, \mu_k)$$

- $b_k, b_k^{\{max\}}$: charge
- c_k : Load Capacity
- v_k^{cur} : Current Node
- μ_k : Unit Distance Energy Consumption

Task



$$\tau_i = (r_i, v_i, w_i, d_i)$$

- r_i : Task Release Time
- v_i : Task Node
- w_i : Cargo Weight
- d_i : Deadline

Charging Station



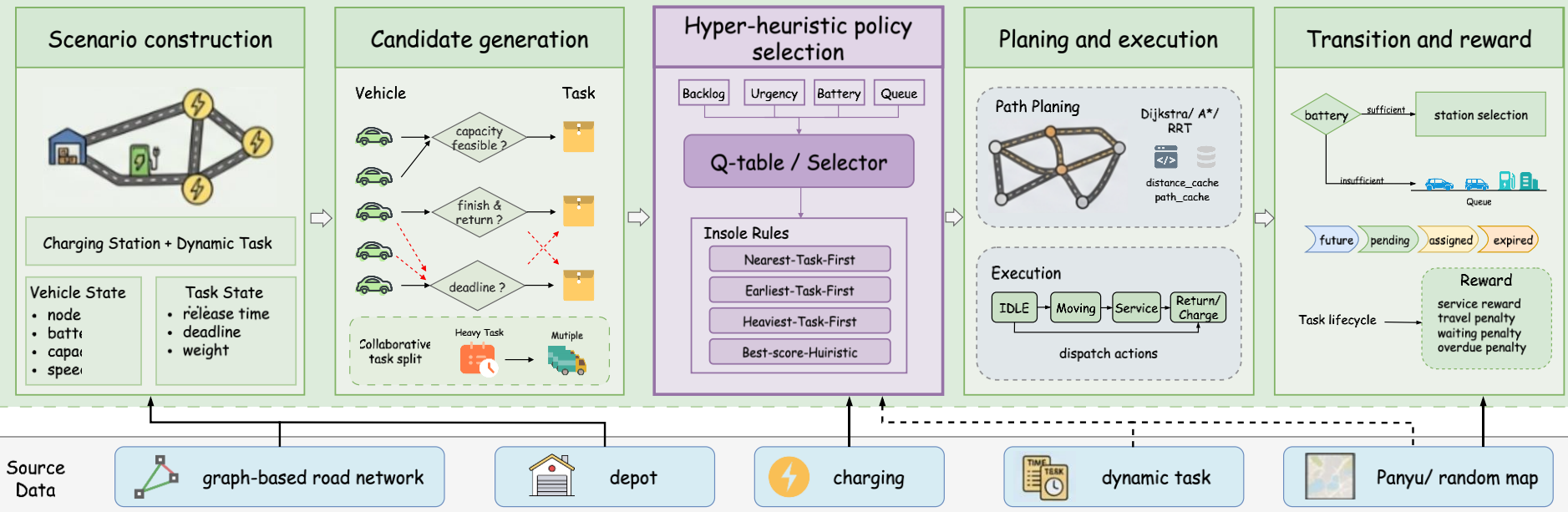
$$S = \{s_1, s_2, \dots, s_m\} \in V$$

Each charging station records: **Number of charging piles/Current occupancy status/Waiting queue length**
Charging rate

Framework

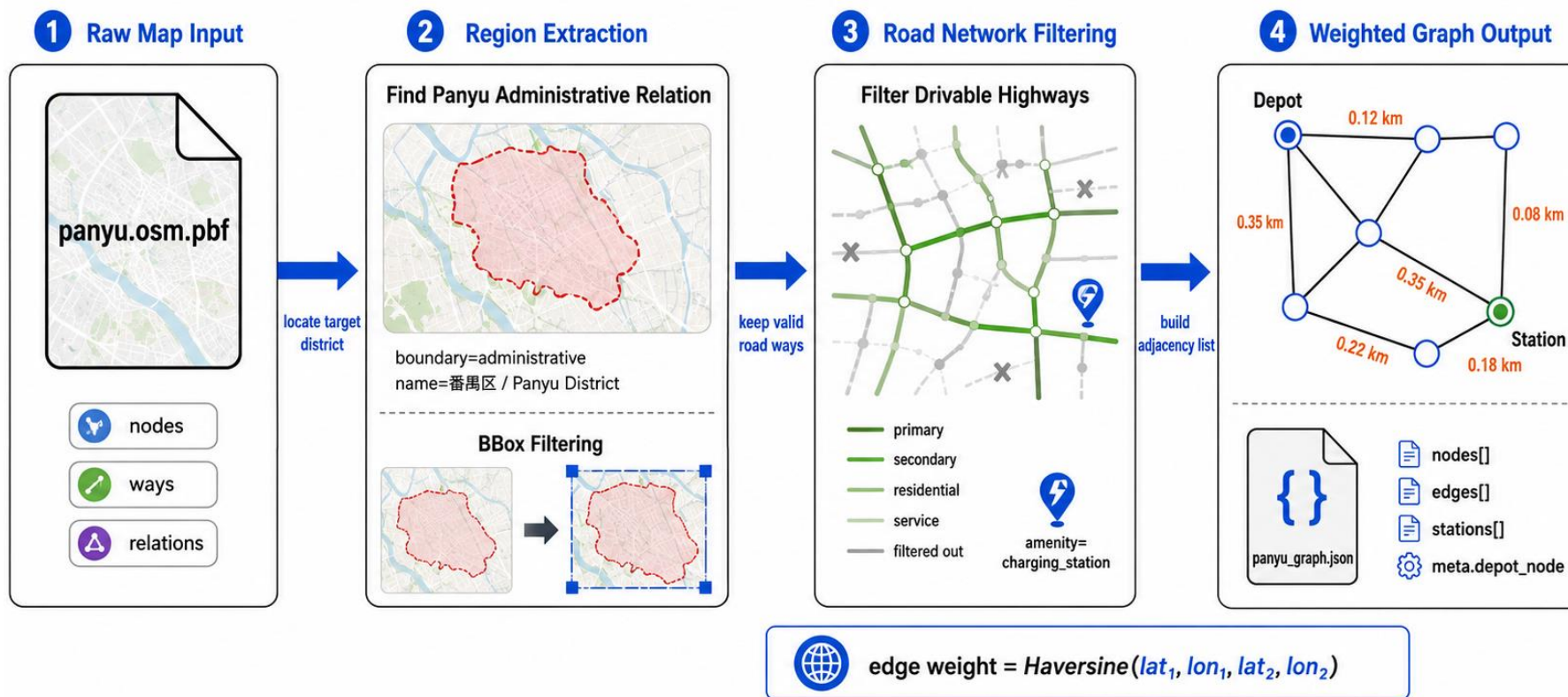


Graph-based Event-Driven Collaborative Scheduling



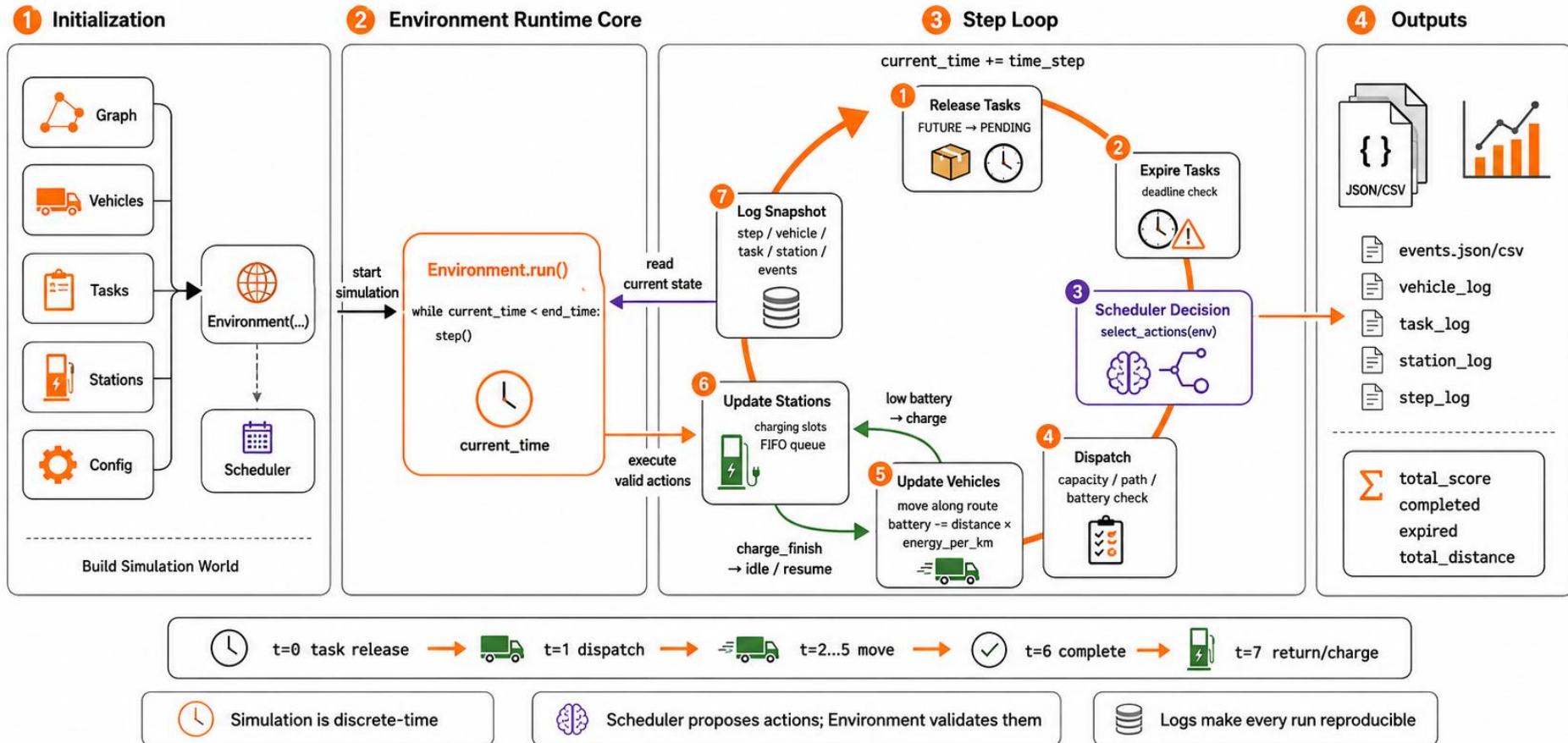
Road Process

Real Map Preprocessing: From OSM Data to Weighted Road Graph

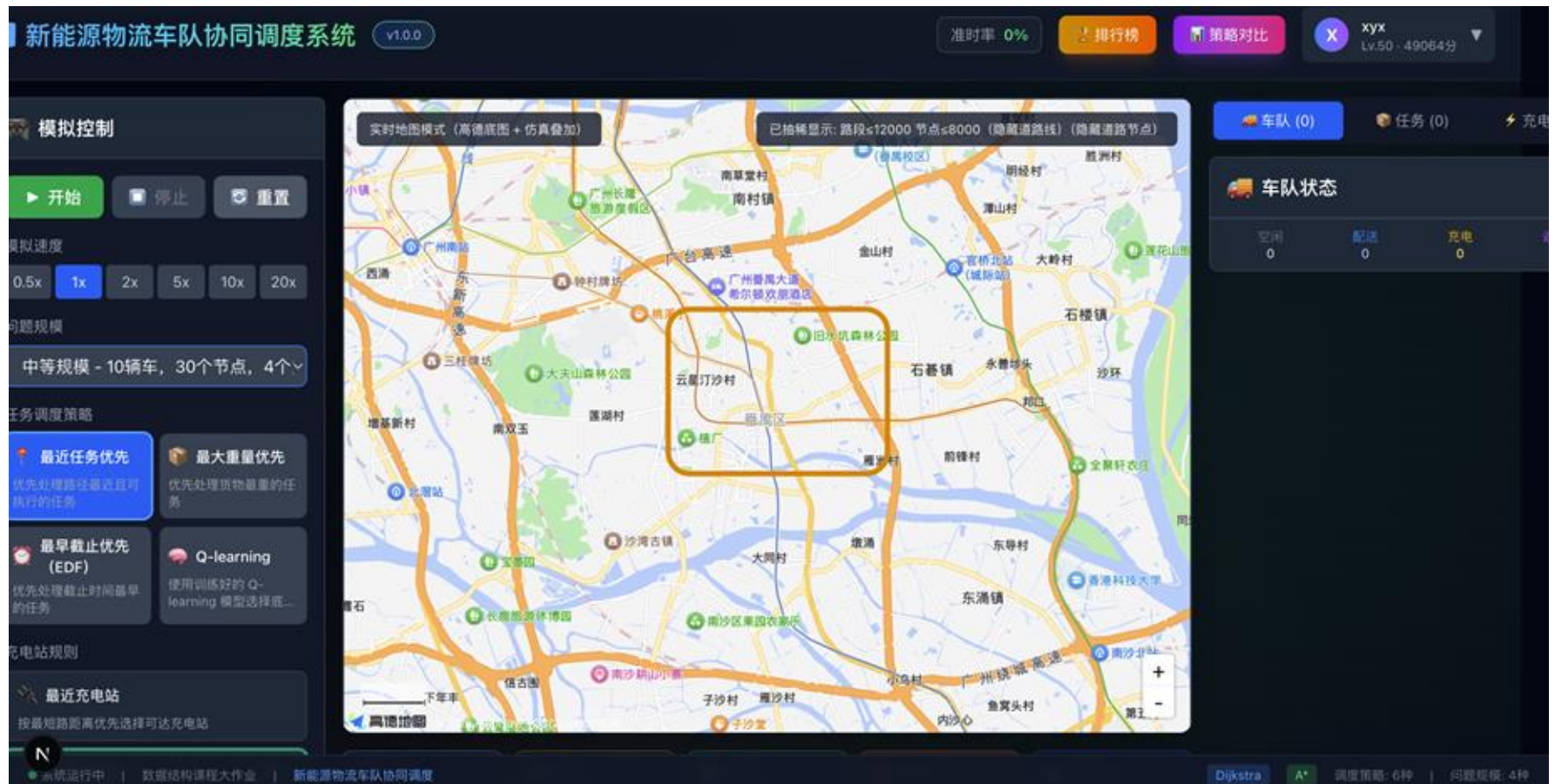


panyu.osm.pbf → Region Extraction → Road Filtering → Weighted Graph

Simulation Engine



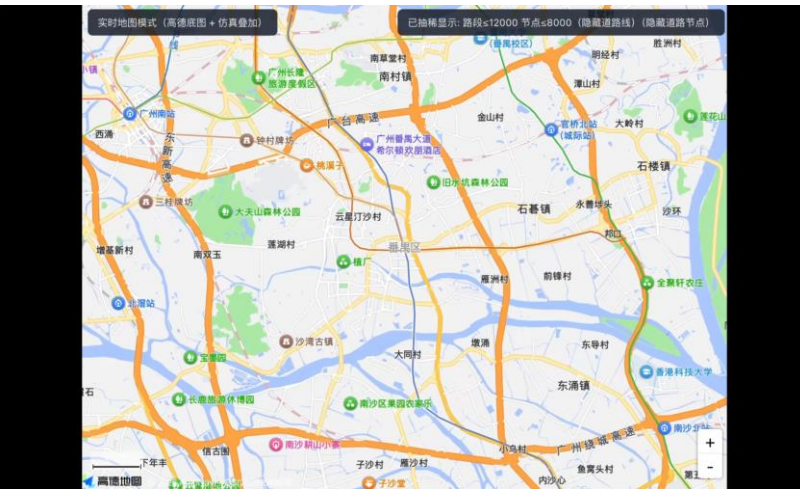
Interactive User Interface



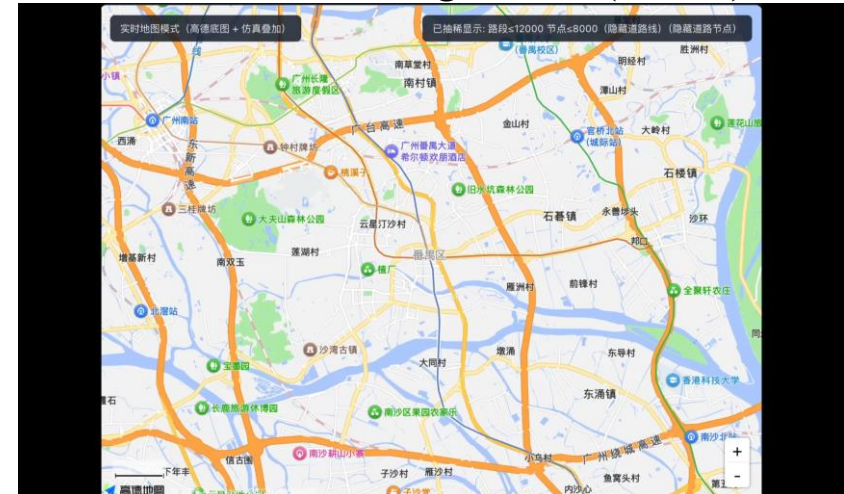
这里写写实现的功能

Dynamic Simulation

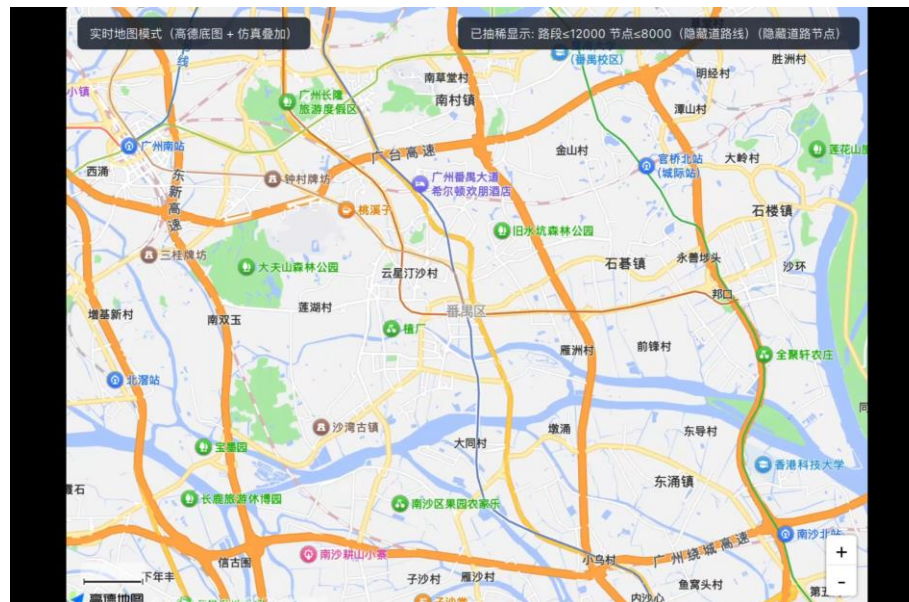
Nearest Task First (NTF)



Maximum-Weight-First (MWF)



Earliest-Deadline-First (EDF)



Path Planning

$$P_k^*(v_k^{\text{cur}}, v) = \arg \min_{P \in \mathcal{P}(v_k^{\text{cur}}, v)} \sum_{e \in P} W(e)$$

Unified Optimization Objective: Find the feasible path with the *minimum cost*

Dijkstra.

Stable Baseline

$$\arg \min \sum_{e \in P} W(e)$$

$$O((V + E) \log V)$$

A star

Heuristic acceleration



$$f(n) = g(n) + h(n)$$

Accumulated Cost Curve Distance

Fast Searching
High Frequency Searching for
large-scale scenery

RRT

Sampling Extension



Fast Extensible
Random Tree Based
on Random Sampling

Shortest Path ✗
Continuous Space ✓
Obstacle Avoidance

Energy Feasible:

$$E_k(P) = \eta_k \cdot C(P)$$

Only when current charge $b_k \geq E_k(P) + E_k(P_{\text{return}})$ Path is executable !

Task Schedule

The three heuristic strategies are essentially different scoring functions under the same framework.

$$\tau_k^* = \arg \min_{\tau_i \in T_t} \phi_k(\tau_i)$$

Nearest Task First (NTF)

$$\tau_k^* = \arg \min_{\tau_i \in T_t} C(P_k^*(v_k^{cu}, v_i))$$

- Short travel distance
- low energy consumption
- fast dispatching response.

Completely ignores deadlines !

Far Away Urgent tasks will be delayed until they expire !

Earliest-Deadline-First (EDF)

$$\tau_k^* = \arg \min_{\tau_i \in T_t} d_i$$

- Directly targets deadline constraints
- fewer expired tasks in small-scale scenarios

Ignores distance and battery level.

Far Away Urgent Task

- ⇒ long distance
- ⇒ more delays
- ⇒ additional charging needs.

Maximum-Weight-First(MWF)

$$\tau_k^* = \arg \max_{\tau_i \in T_t, w_i \leq c_k} w_i$$

- High carrying value per trip,
- suitable for scenarios with dense heavy-load tasks.

considers neither distance nor deadlines !!!

The weakest response to urgent tasks and the highest risk of task expiration.

RL and MILP

Q Learning

State: S

- Backlog : the degree of accumulation of unassigned tasks
- Urgency : the time pressure of current task deadlines
- Battery : the overall battery level of the fleet
- Queue : the queuing or congestion status at charging station

a_t



$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$$

ϵ -greedy



$$a^* = \arg \max_a Q(s_t, a)$$

$$\epsilon \leftarrow \max(\epsilon_{\min}, \rho \cdot \epsilon)$$

MILP

All tasks are known



mixed-integer linear programming problem.



$$\min \sum_{v \in V} \sum_{(i,j) \in A} C_{ij} x_{vij} + \lambda_1 \sum_v \sum_n \text{tardy}_{vn} + \lambda_2 T_{\max}$$



Gurobi

Constraint:

- **Task assignment constraint:** each task is served exactly once by one vehicle.
- **Flow conservation:** vehicle routes must be continuous; if a vehicle enters a node, it must also leave that node.
- **Battery / SOC constraint:** the battery level decreases with driving and increases with charging, and must never become negative.
- **Arrival time / deadline constraint:** the arrival time must not be earlier than the release time; lateness is defined as arrival time minus the deadline.
- **Capacity constraint:** the load carried along the route must not exceed the vehicle capacity

Experiment

Small

Vehicles: 5
Dynamic tasks: 30
Charging stations: 2
Road Nodes: 25
Map size : 30×30 Simulation
horizon: 180

Medium

Vehicles: 10
Dynamic tasks: 100
Charging stations: 4
Road Nodes: 60
Map size : 50×50 Simulation
horizon: 300

large

Vehicles: 20
Dynamic tasks: 300
Charging stations: 8
Road Nodes: 120
Map size : 80×80 Simulation
horizon: 480

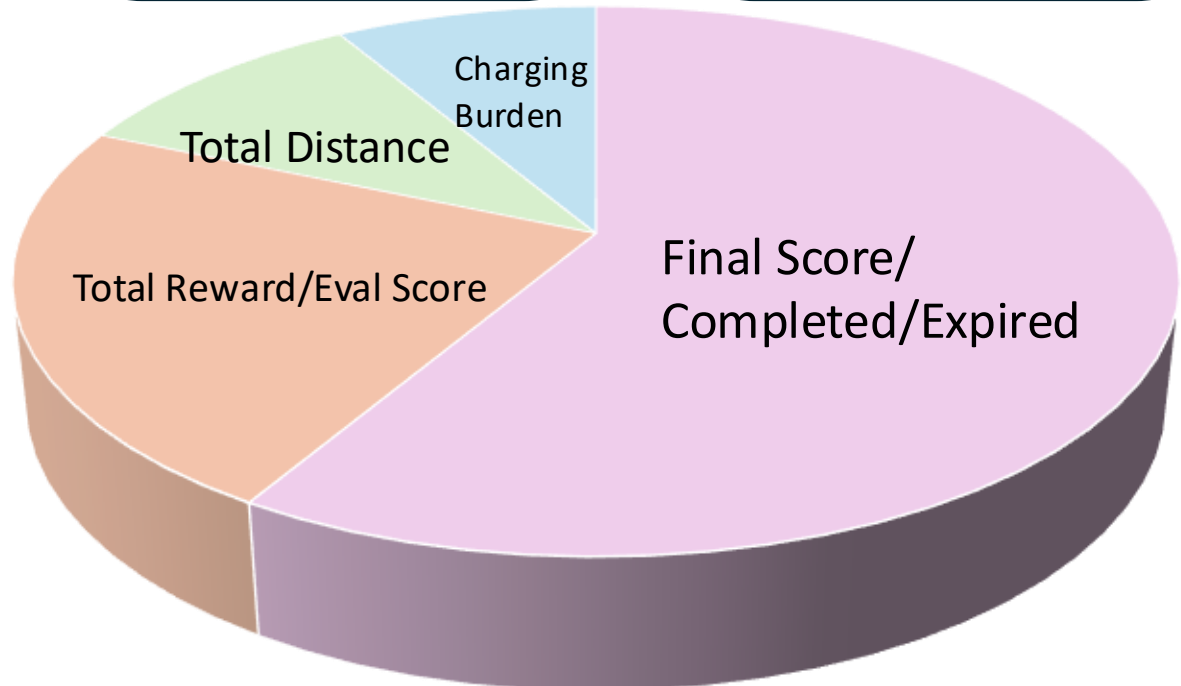
Three Baseline

VS

Q Learning

VS

MILP



Experiment

Nearest is the most robust !

Q-learning performs better in small-scale scenarios !

Charging strategies are more critical for learning-based methods !





THANKS

2026-5-30